

Der 2x-Joker-Algorithmus: 1256.cpp

Yusuf Enes Tamer^{1,2},

Received: September 14, 2025, **Published:** September 15, 2025

¹ Aalen University

²Computer Science Department, HTW-Aalen

Abstract

Es soll ein Algorithmus in der Programmiersprache C++ geschrieben werden. Dabei sind zwei Arrays zur Verfügung. Diese Arrays dürfen nur mit ihren eigenen Indexen zugegriffen werden. Es erlaubt damit den Zugriff nur mit einer Variable, deren Speicheradresse konstant ist. Dieser Algorithmus wurde nach 12 Tagen wieder weiterentwickelt, sodass dieser alle einstelligen Ziffern berechnet und ausgibt.

Introduction

Dieser Algorithmus generiert die Zahlenfolge: 1,2,3,4,5,6. Um diese Zahlenfolge zu generieren, wird ein Datensatz bzw. eine Oracle verwendet, dieser auch gleichzeitig ein Joker-Feld ist, die in diesem Fall folgende Zahlen beinhalten: 1,2,5,6. Diese wird benutzt, nichtsdestotrotz ist die Generierung der Zahlenfolge auf die Anzahl der benutzten Operationen eingeschränkt.

1. 2x Addition
2. 2x Subtraktion
3. 2x direkte Zuweisung

Eine weitere Einschränkung ist, dass das Ausgabefeld und das Joker-Feld nur mit ihren eigenen Indexen zugegriffen werden. Der Algorithmus wird folgendermaßen definiert:

2x-Zuweisung

```
for(i; i < 2; i++)  
{  
  j = i;  
  line[i] = joker[j];  
}
```

Die Zahlen 1,2 werden aus dem Joker-Feld zugewiesen.

1x-Addition

```
line[i] = line[i-2] + line[i-1];  
j++;  
i++,
```

Die Zahl 3 wird im Ausgabefeld generiert.

2x-Subtraktion

```
for( j; j<=4; j++)  
{
```

```
  line[i] = joker[j] - joker[j - j];
```

Die Zahlen 4,5 werden aus dem Joker-Feld herausgerechnet.

1x-Addition

```
if( j == 4)
```

```
{  
  line[i] = line[i-1] + line[i - j - 1];  
}
```

```
i++;
```

Die Zahl 6 wird im Ausgabefeld erzeugt, ohne den Joker zu verwenden.

Schlussfolgerung

Der Algorithmus ist optimal, da es sich bei der Verwendung der Operatoren um eine Gleichverteilung handelt:

$$2z + 2s + (1a + 1a)$$

Der Algorithmus wurde so programmiert, dass es erweiterbar ist. Bei der Weiterprogrammierung wurde für den Objektorientierten-Paradigma entschieden. Dieser ist auf der nächsten Seite zu finden:

<https://github.com/yusuta-gba/>

[Cryptographic-Algorithms/blob/main/1256.cpp](https://github.com/yusuta-gba/Cryptographic-Algorithms/blob/main/1256.cpp)

Objektorientierte-Programmierung des 1256-Algorithmus

Das Hauptprogramm sieht folgendermaßen aus:

```
loop * l1 = new loop(0,1);
loop * l2 = new loop(3,5);
int line[9];
int joker[] = { 1, 2, 5, 6};
l1->assign(line, joker);
line[l1->index] = line[l1->index-2] + line[l1->index-1];
l1->incl();
l2->index = l1->getIndex();
l2->subAdd(line, joker);
loop * l3 = new loop(l2->getIndex(), l2->getIndex() +
(l2->getEnd() - l2->getStart()));
l3->moduloAdd(line, joker);
```

Der Ansatz ist, dass alle FOR-Schleifen zu einer Klasse gehören und davon der Objekt ist. Damit hat ein Schleifen-Objekt drei Attribute:

- Startwert
- Endwert
- dynamischer Wert

Das Schleifen-Objekt wird mit einem Konstruktor erstellt und dabei werden 2 Parameter übergeben. Dieser initialisieren den Start- und den Endwert. Um die Zahlen bis zu 9 rechnen zu lassen, wird ein weiteres Schleifen-Objekt generiert. Dieser ist im Code **loop3** und werden mit der vorherigen berechneten Zahlen initialisiert. Die Zahlengenerierung erfolgt in der richtigen Reihenfolge und damit ist die Umsetzung davon möglich, dass das Computer-Programm die Grenzwerte erhält, somit mit einem relativen Offset alle Zahlen in diesem Intervall zugreifbar sind. Danach können Funktionsgleichungen aufgestellt werden, sodass die Inkremente von der nächsten Zahlenreihe erfolgen.

```
line[9] = line[0] + line[8];
for(int t = 1; t <= 10; t++)
{
line[t + 9] = line[9] + line[t];
}
```

```
https://github.com/yusuta-gba/
Cryptographic-Algorithms/blob/main/125689.
cpp
https://github.com/yusuta-gba/
Cryptographic-Algorithms/blob/main/loop.cpp
https://github.com/yusuta-gba/
Cryptographic-Algorithms/blob/main/Counter.
cpp
```